| Name: Jal Bafana | Roll no: K005 |
|---|---|
| Btech. Cyber Security (Sem-6) | Batch: K1 |

# Experiment No. 9

**Aim**: Recognizing Packed Malware and unpack it with various tools
**Title**: Malware obfuscation techniques.
**Part A:**
Packing and obfuscation are common techniques used in malware to evade pattern-based detection and to give Malware analyst/Reverse engineer a hard time reaching to the malicious content. These techniques are not only used by malicious file these are also used by some legitimate software to prevent reverse engineer getting to the original source code or to crack the software. In this blog we will talk about both the technique one by one with the help of some packed and obfuscated files.

**Packing**
Packing is a subset of obfuscation. A packer is a tool that modifies the formatting of code by compressing or encrypting the data.
Though often used to delay the detection of malicious code, there is still legitimate use for packing. Some legitimate use includes protecting intellectual property or other sensitive data from being copied.
A stub is a small portion of code that contains the decryption or decompression agent used to decrypt the packed file.

**The packing process consists of:**

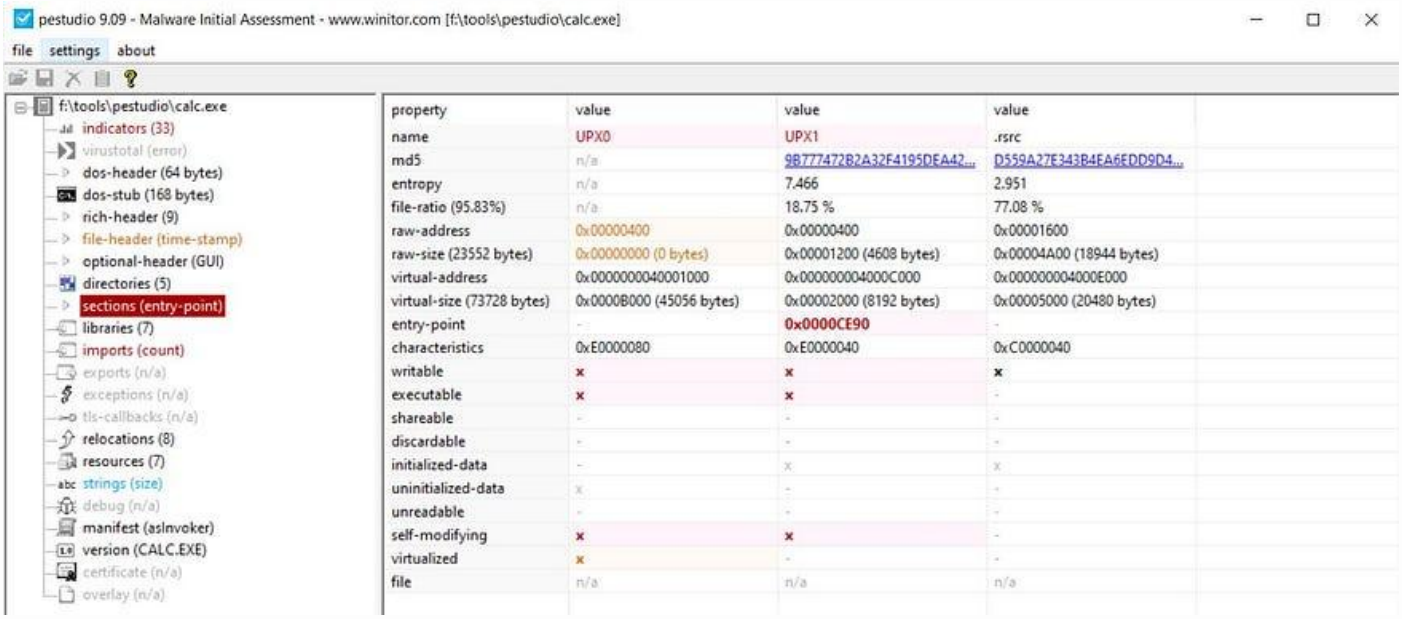
- The original code is uploaded into the packer tool and goes through the packer process to compress or encrypt the data.
- The original portable executable header (PE header, which consists of executable image and object files), and original code are compressed or encrypted and stored in the packed section of the new executable
- The packed file consists of:
    - New PE header
    - Packed section(s)
    - Decompression stub — used to unpack the code.
- During the packing process, the original entry point is relocated/obfuscated in the packed section. This is important for anyone trying to analyse the code. This process makes identifying the import address table (IAT) and original entry point difficult.
- The decompression stub is used to unpack the code upon delivery.

Some malware creators use custom packers, but commercial/open-source packers are also used. Some popular packers include:

- UPX
- Themida
- The Enigma Protector
- VMProtect
- Obsidium
- MPRESS
- Exe Packer 2.300
- ExeStealth

**How does UPX works?**
Upx is commonly used packer so let's talk about how it works. Upx create an executable which contains 3 main part which are:
**Stub:** This contains the code to decompress the compressed executable this is the entry point of the packed executable.

**Compressed executable:** This is the original executable that is compressed with upx packer which going to be decompressed by unpacking stub.
**Empty space:** Empty space is used to store the unpacked executable. If you notice all the section in packed executable in pe studio you will get a section (generally UPX0) which have 0 raw-size and large virtual-size which means size of that section is going to be larger during runtime which makes sense because the packed executable is going to be unpacked and executed so it need space to reside.




| property | value | value | value |
|---|---|---|---|
| name | UPX0 | UPX1 | .rsrc |
| md5 | n/a | 9B777472B2A32F4195DEA42... | D559A27E343B4EA6EDD9D4... |
| entropy | n/a | 7.466 | 2.951 |
| file-ratio (95.83%) | n/a | 18.75 % | 77.08 % |
| raw-address | 0x00000400 | 0x00000400 | 0x00001600 |
| raw-size (23552 bytes) | 0x00000000 (0 bytes) | 0x00001200 (4608 bytes) | 0x00004A00 (18944 bytes) |
| virtual-address | 0x0000000040001000 | 0x000000004000C000 | 0x000000004000E000 |
| virtual-size (73728 bytes) | 0x0000B000 (45056 bytes) | 0x00002000 (8192 bytes) | 0x00005000 (20480 bytes) |
| entry-point | - | 0x0000CE90 | - |
| characteristics | 0xE0000080 | 0xE0000040 | 0xC0000040 |
| writable | x | x | x |
| executable | x | x | - |
| shareable | - | - | - |
| discardable | - | - | - |
| initialized-data | - | x | x |
| uninitialized-data | x | - | - |
| unreadable | - | - | - |
| self-modifying | x | x | - |
| virtualized | x | - | - |
| file | n/a | n/a | n/a |

Sections of a packed executable

**How unpacking works?**
Good thing about packing is if you want to execute a code you will have to unpack the code because packed code cannot be executed. If a malware is packed it will have to unpack itself to run so a reverse engineer knows he will just have to find the unpacking method to get to the malicious code, it can be an unpacking code present in malware itself or a C2 command that does that. Above we talked about packed file now lets talk how does this file will get unpacked. The file which are packed with UPX gets unpacked when we execute them because of unpacking stub. The stub not just only unpacks the executable but also do the following:

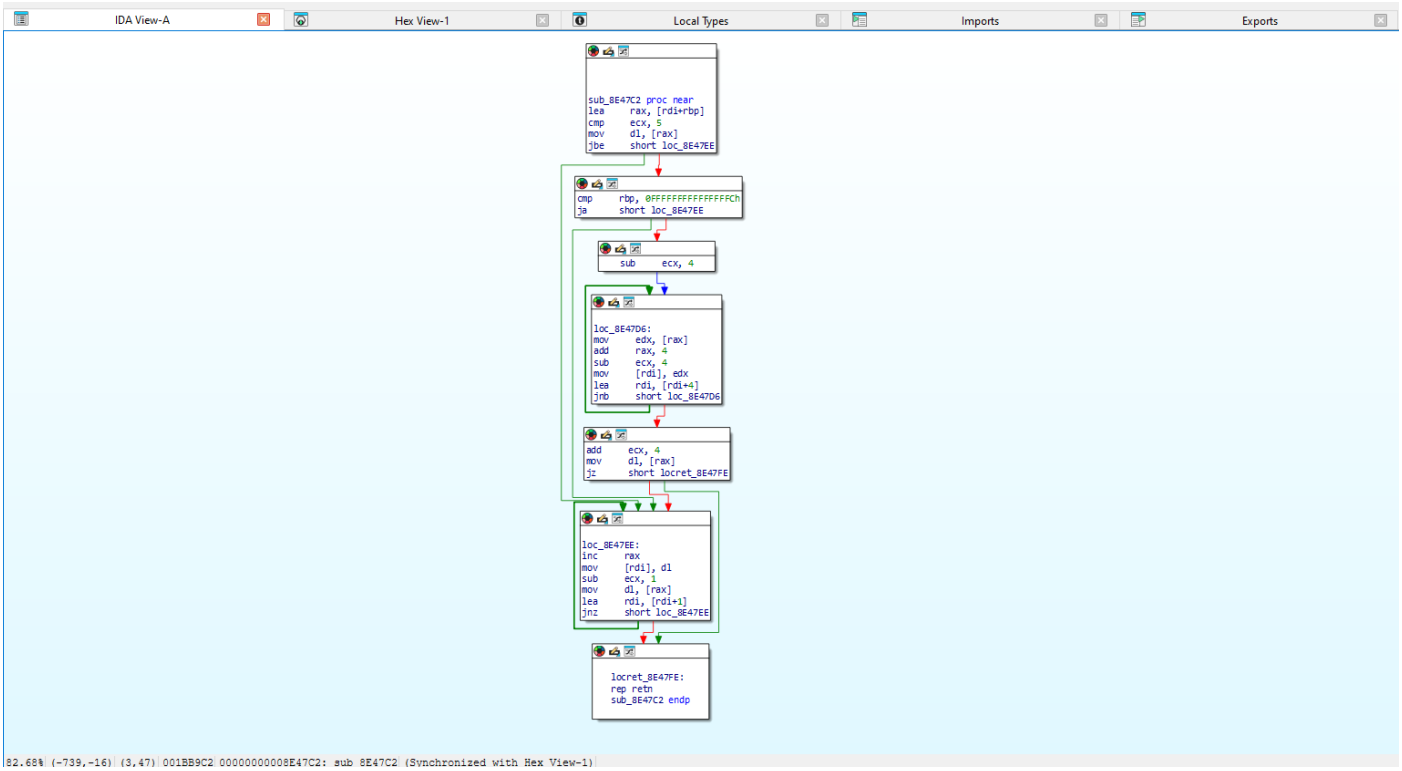
1. Extracting packed file
2. Resolving imports
3. Restoring file permissions
4. Jumping to the Original entry point (Unpacked executable's entry point)

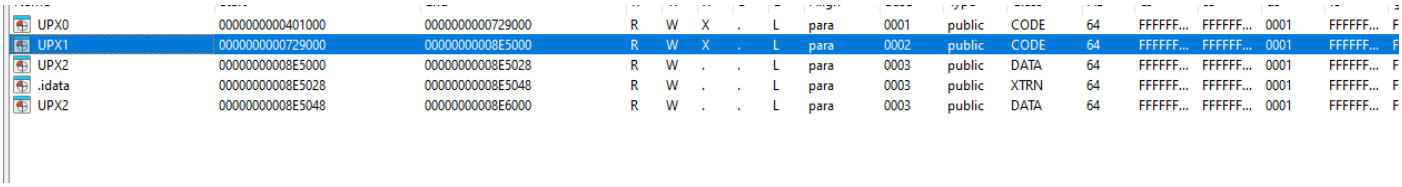
After unpacking the original code into the UPX0 section imports of the original files are resolved if you notice import section in pe studio of a packed you will find very a smaller number of imports than the actual file uses these imports are resolved after unpacking. After restoring the file permission using VirtualProtect, execution flow is then transferred to the entry point of the original file. We can use UPX itself to unpack a upx packed file or we can just do it manually with the help of debugger.

**Part B:**

1. Refer
2. https://medium.com/@dbragetti/unpacking-malware-685de7093e5
3. Pack a exe file using UPX tool
4. Unpack a UPX packed file
5. Analyse header details of original and packed exe files using IDA pro.
6. Analyse number of imports in packed vs unpacked using ollydb
7. Conclude with output screenshots.

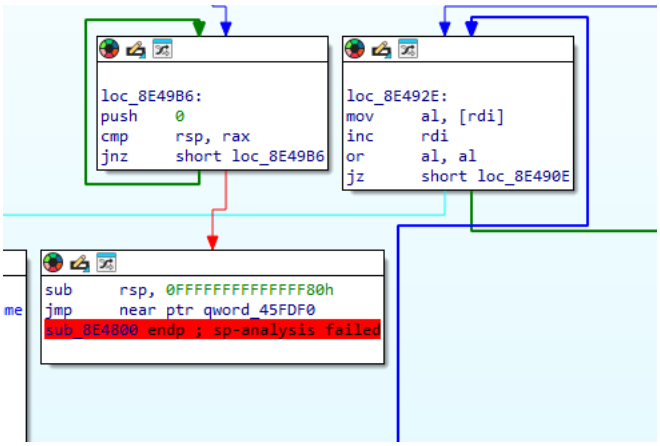
Packed EXE File




Importing

| Address | Ordinal | Name |
|---|---|---|
| KERNEL32 | | |
| 00000000008E5028 | | LoadLibraryA |
| 00000000008E5030 | | ExitProcess |
| 00000000008E5038 | | GetProcAddress |
| 00000000008E5040 | | VirtualProtect |




The analysis has failed.

```
UPX0:0000000000401000                 .model flat
UPX0:0000000000401000
UPX0:0000000000401000 ; ===========================================================================
UPX0:0000000000401000
UPX0:0000000000401000 ; Segment type: Pure code
UPX0:0000000000401000 ; Segment permissions: Read/Write/Execute
UPX0:0000000000401000 UPX0            segment para public 'CODE' use64
UPX0:0000000000401000                 assume cs:UPX0
UPX0:0000000000401000                 ;org 401000h
UPX0:0000000000401000                 assume es:nothing, ss:nothing, ds:UPX0, fs:nothing, gs:nothing
UPX0:0000000000401000                 dq 0BDBEh dup(?)
UPX0:000000000045FDF0 qword_45FDF0    dq 59242h dup(?)        ; CODE XREF: sub_8E4800+1C1↓j
UPX0:000000000045FDF0 UPX0            ends
UPX0:000000000045FDF0
UPX1:0000000000729000 ; Section 2. (virtual address 00329000)
UPX1:0000000000729000 ; Virtual size                  : 001BC000 (1818624.)
UPX1:0000000000729000 ; Section size in file          : 001BBA00 (1817088.)
UPX1:0000000000729000 ; Offset to raw data for section: 00000200
UPX1:0000000000729000 ; Flags E0000040: Data Executable Readable Writable
UPX1:0000000000729000 ; Alignment     : default
UPX1:0000000000729000 ; ===========================================================================
UPX1:0000000000729000
UPX1:0000000000729000 ; Segment type: Pure code
UPX1:0000000000729000 ; Segment permissions: Read/Write/Execute
UPX1:0000000000729000 UPX1            segment para public 'CODE' use64
UPX1:0000000000729000                 assume cs:UPX1
UPX1:0000000000729000                 ;org 729000h
UPX1:0000000000729000                 assume es:nothing, ss:nothing, ds:UPX0, fs:nothing, gs:nothing
UPX1:0000000000729000 a395            db '3.95',0
```

After seeing target address,
Found out we are at the end of UPX0 section,
Disassembler was not able to decode any instruction in it.

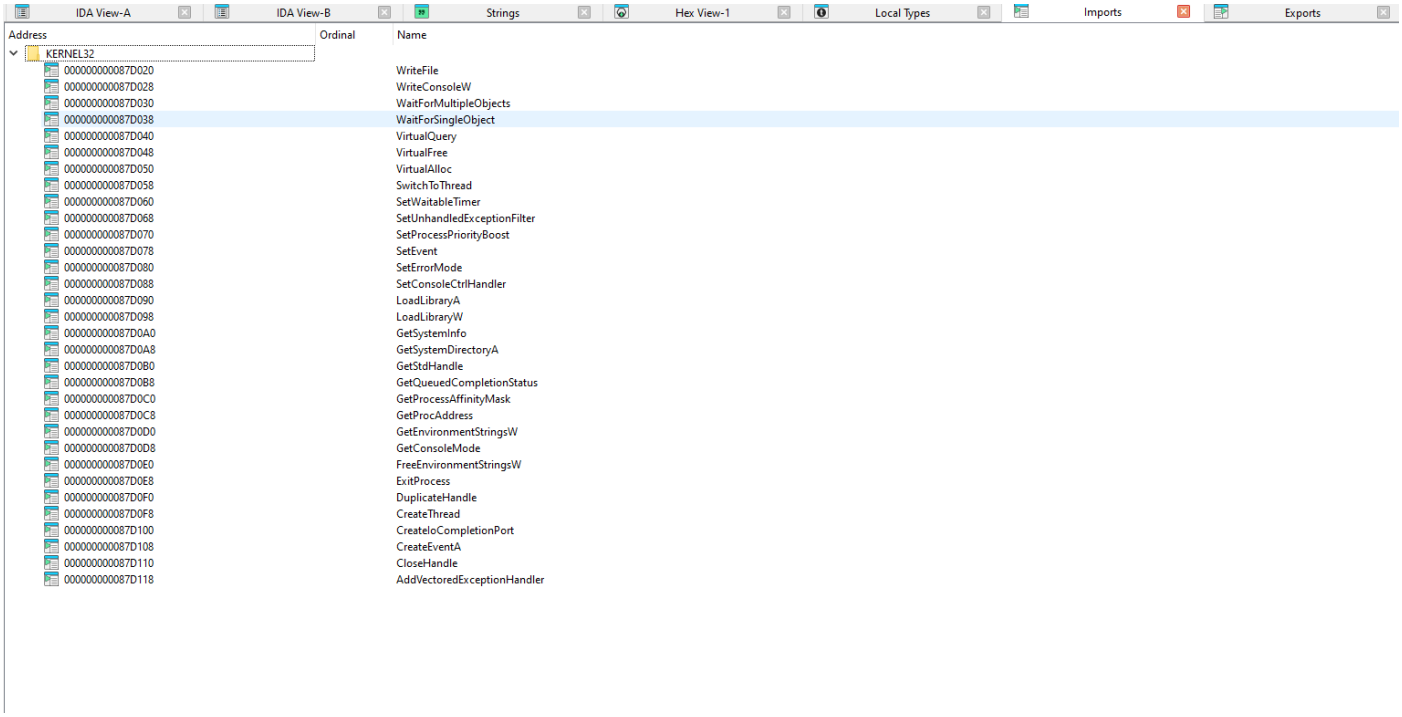
```
UPX0:000000000045FDF0 qword_45FDF0    dq 59242h dup(?)        ; CODE XREF: sub_8E4800+1C1↓j
UPX0:000000000045FDF0 UPX0            ends
UPX0:000000000045FDF0
UPX1:0000000000729000 ; Section 2. (virtual address 00329000)
UPX1:0000000000729000 ; Virtual size                  : 001BC000 (1818624.)
UPX1:0000000000729000 ; Section size in file          : 001BBA00 (1817088.)
UPX1:0000000000729000 ; Offset to raw data for section: 00000200
UPX1:0000000000729000 ; Flags E0000040: Data Executable Readable Writable
UPX1:0000000000729000 ; Alignment     : default
UPX1:0000000000729000 ; ===========================================================================
UPX1:0000000000729000
UPX1:0000000000729000 ; Segment type: Pure code
UPX1:0000000000729000 ; Segment permissions: Read/Write/Execute
UPX1:0000000000729000 UPX1            segment para public 'CODE' use64
```

Exe packed by UPX.

Unpacked exe file (RedlineStealer)

```
.text:0000000000401000 ;
.text:0000000000401000 ; +-------------------------------------------------------------------------+
.text:0000000000401000 ; |      This file was generated by The Interactive Disassembler (IDA)      |
.text:0000000000401000 ; |           Copyright (c) 2026 Hex-Rays, <support@hex-rays.com>           |
.text:0000000000401000 ; |                         Freeware version                                |
.text:0000000000401000 ; +-------------------------------------------------------------------------+
.text:0000000000401000 ;
.text:0000000000401000 ; Input SHA256 : 12E2635032D6FD6E23DEFC1A3A2B79C2166204870A2F3816E04460BD725B1F58
.text:0000000000401000 ; Input MD5    : 039D15E673E64C4B8CE2EB186C644CD4
.text:0000000000401000 ; Input CRC32  : CA10C5CC
.text:0000000000401000 ; Compiler     : GNU C++
.text:0000000000401000
.text:0000000000401000 ; File Name   : C:\Users\heroic\Desktop\packing-malware\3215decffc40b3257ebeb9b6e5c81c45e298a020f33ef90c9418c153c6071b36\3215decffc40b3257ebeb
.text:0000000000401000 ; Format      : Portable executable for AMD64 (PE)
.text:0000000000401000 ; Imagebase   : 400000
.text:0000000000401000 ; Timestamp   : 00000000 (Thu Jan 01 00:00:00 1970 UTC)
.text:0000000000401000 ; Section 1. (virtual address 00001000)
.text:0000000000401000 ; Virtual size                  : 0023DD13 (2350355.)
.text:0000000000401000 ; Section size in file          : 0023DE00 (2350592.)
.text:0000000000401000 ; Offset to raw data for section: 00000600
.text:0000000000401000 ; Flags 60000060: Text Data Executable Readable
.text:0000000000401000 ; Alignment     : default
.text:0000000000401000
.text:0000000000401000                 .686p
.text:0000000000401000                 .mmx
.text:0000000000401000                 .model flat
.text:0000000000401000 .intel_syntax noprefix
.text:0000000000401000
.text:0000000000401000 ; ===========================================================================
.text:0000000000401000
.text:0000000000401000 ; Segment type: Pure code
.text:0000000000401000 ; Segment permissions: Read/Execute
.text:0000000000401000 _text           segment para public 'CODE' use64
.text:0000000000401000                 assume cs:_text
.text:0000000000401000                 ;org 401000h
.text:0000000000401000                 assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing
.text:0000000000401000
.text:0000000000401000 ; =============== S U B R O U T I N E =======================================
.text:0000000000401000
.text:0000000000401000
.text:0000000000401000 go_buildid      proc near               ; DATA XREF: .rdata:000000000067A880↓o
.text:0000000000401000                                         ; .rdata:000000000067A8E0↓o ...
.text:0000000000401000                 jmp     qword ptr [rax]
.text:0000000000401000 go_buildid      endp
.text:0000000000401000
.text:0000000000401000 ; ---------------------------------------------------------------------------
.text:0000000000401002                 dw 6F47h, 6220h, 6975h
.text:0000000000401008                 dq 22203A444920646Ch, 6257744C77526E39h, 486570534671714Fh
.text:0000000000401020                 dq 507A462F746F4931h, 477053762D506D4Ah, 4643394459733374h
.text:0000000000401038                 dq 373444586A4D2F49h, 69785F674A646F48h, 632F653871446661h
00000600 0000000000401000: go_buildid (Synchronized with Hex View-1)
```

Imports


IDA View-A
IDA View-B
Strings
Hex View-1
Local Types
Imports
Exports
Address
Ordinal
Name
KERNEL32
000000000087D020 WriteFile
000000000087D028 WriteConsoleW
000000000087D030 WaitForMultipleObjects
000000000087D038 WaitForSingleObject
000000000087D040 VirtualQuery
000000000087D048 VirtualFree
000000000087D050 VirtualAlloc
000000000087D058 SwitchToThread
000000000087D060 SetWaitableTimer
000000000087D068 SetUnhandledExceptionFilter
000000000087D070 SetProcessPriorityBoost
000000000087D078 SetEvent
000000000087D080 SetErrorMode
000000000087D088 SetConsoleCtrlHandler
000000000087D090 LoadLibraryA
000000000087D098 LoadLibraryW
000000000087D0A0 GetSystemInfo
000000000087D0A8 GetSystemDirectoryA
000000000087D0B0 GetStdHandle
000000000087D0B8 GetQueuedCompletionStatus
000000000087D0C0 GetProcessAffinityMask
000000000087D0C8 GetProcAddress
000000000087D0D0 GetEnvironmentStringsW
000000000087D0D8 GetConsoleMode
000000000087D0E0 FreeEnvironmentStringsW
000000000087D0E8 ExitProcess
000000000087D0F0 DuplicateHandle
000000000087D0F8 CreateThread
000000000087D100 CreateIoCompletionPort
000000000087D108 CreateEventA
000000000087D110 CloseHandle
000000000087D118 AddVectoredExceptionHandler